Module `harfbuzz`

Lua bindings to Harfbuzz.

- Wiki
- Source on Github
- API Coverage Status

Info:

- **Copyright**: 2016
- **License**: MIT
- **Author**: Deepak Jois <deepak.jois@gmail.com>

Functions

version ()	Wraps `hb_version`.
shape (font, buffer[, options])	Wraps `hb_shape`.

Class Blob

Blob.new (data)	Wraps `hb_blob_create`.
Blob.new_from_file (filename)	Wraps `hb_blob_create_from_file`.
Blob:get_length ()	Wraps `hb_blob_get_length`.
Blob:get_data ()	Wraps `hb_blob_get_data`.

Class Face

Face.new_from_blob (blob[, font_index=0])	Wraps `hb_face_create`.
Face.new (file[, font_index=0])	Create a new `Face` from a file.
Face:blob ()	Wraps `hb_face_reference_blob`.
Face:get_name (name_id[, lang])	Wraps `hb_ot_name_get_utf8`.
Face:collect_unicodes ()	Wraps `hb_face_collect_unicodes`.
Face:get_glyph_count ()	Wraps `hb_face_get_glyph_count`.
Face:get_table (tag)	Wraps `hb_face_reference_table`.
Face:get_table_tags ()	Wraps `hb_face_get_table_tags`.
Face:get_upem ()	Wraps `hb_face_get_upem`.
Face:ot_color_has_palettes ()	Wraps `hb_ot_color_has_palettes`.
Face:ot_color_palette_get_count ()	Wraps `hb_ot_color_palette_get_count`.
Face:ot_color_palette_get_colors ([palette_index=1])	Wraps `hb_ot_color_palette_get_colors`.
Face:ot_color_has_layers ()	Wraps `hb_ot_color_has_layers`.
Face:ot_color_glyph_get_layers (glyph)	Wraps `hb_ot_color_glyph_get_layers`.
Face:ot_color_has_png ()	Wraps `hb_ot_color_has_png`.
Face:ot_color_has_svg ()	Wraps `hb_ot_color_has_svg`.
Face:ot_color_glyph_get_svg (glyph)	Wraps `hb_ot_color_glyph_reference_svg`.
Face:ot_var_has_data ()	Wraps `hb_ot_var_has_data`.
Face:ot_var_find_axis_info (tag)	Wraps `hb_ot_var_find_axis_info`.
Face:ot_var_get_axis_infos ([start=1[, stop=-1]])	Wraps `hb_ot_var_get_axis_infos`.
Face:ot_var_named_instance_get_infos ([start=1[, stop=-1]])	Wraps `hb_ot_var_get_named_instance_count` and related functions.
Face:ot_var_named_instance_get_design_coords (index)	Wraps `hb_ot_var_named_instance_get_design_coords`.
Face:ot_var_normalize_variations (...)	Wraps `hb_ot_var_normalize_variations`.
Face:ot_var_normalize_coords (...)	Wraps `hb_ot_var_normalize_coords`.
Face:ot_layout_get_script_tags (table_tag)	Wraps `hb_ot_layout_table_get_script_tags`.
Face:ot_layout_get_language_tags (table_tag, script_index)	Wraps `hb_ot_layout_script_get_language_tags`.
Face:ot_layout_get_feature_tags (table_tag, script_index, language_index)	Wraps `hb_ot_layout_language_get_feature_tags`.
Face:ot_layout_find_script (table_tag, script_tag)	Wraps `hb_ot_layout_table_find_script`.
Face:ot_layout_find_language (table_tag, script_index, language_tag)	Wraps `hb_ot_layout_script_find_language`.
Face:ot_layout_find_feature (table_tag, script_index, language_index, feature_tag)	Wraps `hb_ot_layout_language_find_feature`.

Class Font

Font.new (face)	Wraps `hb_font_create`, and sets up some defaults for scale and shaping functions.
Font:get_scale ()	Wraps `hb_font_get_scale`.
Font:set_scale (x_scale, y_scale)	Wraps `hb_font_set_scale`.
Font:get_h_extents ()	Wraps `hb_font_get_h_extents`.
Font:get_v_extents ()	Wraps `hb_font_get_v_extents`.
Font:get_glyph_extents (glyph)	Wraps `hb_font_get_glyph_extents`.
Font:get_glyph_name (glyph)	Wraps `hb_font_get_glyph_name`.
Font:get_glyph_from_name (name)	Wraps `hb_font_get_glyph_from_name`.
Font:get_glyph_h_advance (glyph)	Wraps `hb_font_get_glyph_h_advance`.
Font:get_glyph_v_advance (glyph)	Wraps `hb_font_get_glyph_v_advance`.
Font:get_nominal_glyph (codepoint.)	Wraps `hb_font_get_nominal_glyph`.
Font:style_get_value (tag)	Wraps `hb_style_get_value`.
Font:ot_color_glyph_get_png (glyph)	Wraps `hb_ot_color_glyph_get_png`.
Font:ot_metrics_get_position (tag)	Wraps `hb_ot_metrics_get_position`.
Font:ot_metrics_get_variation (tag)	Wraps `hb_ot_metrics_get_variation`.
Font:ot_metrics_get_x_variation (tag)	Wraps `hb_ot_metrics_get_x_variation`.
Font:ot_metrics_get_y_variation (tag)	Wraps `hb_ot_metrics_get_y_variation`.
Font:set_variations (...)	Wraps `hb_font_set_variations`.
Font:set_var_coords_design (...)	Wraps `hb_font_set_var_coords_design`.
Font:set_var_coords_normalized (...)	Wraps `hb_font_set_var_coords_normalized`.
Font:set_var_named_instance (index)	Wraps `hb_font_set_var_named_instance`.
Font:get_var_coords_normalized ()	Wraps `hb_font_get_var_coords_normalized`.

Class Buffer

Buffer.new ()	Wraps `hb_buffer_create`.
Buffer:add (codepoint, cluster)	Wraps `hb_buffer_add`.
Buffer:add_utf8 (text[, item_offset=0[, item_length=-1]])	Wraps `hb_buffer_add_utf8`.
Buffer:add_codepoints (text[, item_offset=0[, item_length=-1]])	Wraps `hb_buffer_add_codepoints`.
Buffer:set_direction (dir)	Wraps `hb_buffer_set_direction`.
Buffer:get_direction ()	Wraps `hb_buffer_get_direction`.
Buffer:set_script (script)	Wraps `hb_buffer_set_script`.
Buffer:get_script ()	Wraps `hb_buffer_get_script`.
Buffer:set_language (lang)	Wraps `hb_buffer_set_language`.
Buffer:get_language ()	Wraps `hb_buffer_get_language`.
Buffer:set_flags (flags)	Wraps `hb_buffer_set_flags`.
Buffer:get_flags ()	Wraps `hb_buffer_get_flags`.
Buffer:set_cluster_level (level)	Wraps `hb_buffer_set_cluster_level`.
Buffer:get_cluster_level ()	Wraps `hb_buffer_get_cluster_level`.
Buffer:set_invisible_glyph (glyph)	Wraps `hb_buffer_set_invisible_glyph`.
Buffer:get_invisible_glyph ()	Wraps `hb_buffer_get_invisible_glyph`.
Buffer:set_replacement_codepoint (codepoint)	Wraps `hb_buffer_set_replacement_codepoint`.
Buffer:get_replacement_codepoint ()	Wraps `hb_buffer_get_replacement_codepoint`.
Buffer:get_length ()	Wraps `hb_buffer_get_length`.
Buffer:guess_segment_properties ()	Wraps `hb_buffer_guess_segment_properties`.
Buffer:reverse ()	Wraps `hb_buffer_reverse`.
Buffer:clear_contents ()	Wraps `hb_buffer_clear_contents`.
Buffer:reset ()	Wraps `hb_buffer_reset`.
Buffer:pre_allocate (size)	Wraps `hb_buffer_pre_allocate`.
Buffer:get_glyphs ()	Helper method to get shaped glyph data.

Cluster Levels

Buffer.CLUSTER_LEVEL_MONOTONE_GRAPHEMES	Wraps `HB_BUFFER_CLUSTER_LEVEL_MONOTONE_GRAPHEMES`.
Buffer.CLUSTER_LEVEL_MONOTONE_CHARACTERS	Wraps `HB_BUFFER_CLUSTER_LEVEL_MONOTONE_CHARACTERS`.
Buffer.CLUSTER_LEVEL_CHARACTERS	Wraps `HB_BUFFER_CLUSTER_LEVEL_CHARACTERS`.
Buffer.CLUSTER_LEVEL_DEFAULT	Wraps `HB_BUFFER_CLUSTER_LEVEL_DEFAULT`.

Buffer Flags

Buffer.FLAG_DEFAULT	Wraps `HB_BUFFER_FLAG_DEFAULT`.
Buffer.FLAG_BOT	Wraps `HB_BUFFER_FLAG_BOT`.
Buffer.FLAG_EOT	Wraps `HB_BUFFER_FLAG_EOT`.
Buffer.FLAG_PRESERVE_DEFAULT_IGNORABLES	Wraps `HB_BUFFER_FLAG_PRESERVE_DEFAULT_IGNORABLES`.
Buffer.FLAG_REMOVE_DEFAULT_IGNORABLES	Wraps `HB_BUFFER_FLAG_REMOVE_DEFAULT_IGNORABLES`.
Buffer.FLAG_DO_NOT_INSERT_DOTTED_CIRCLE	Wraps `HB_BUFFER_FLAG_DO_NOT_INSERT_DOTTED_CIRCLE`.
Buffer.FLAG_VERIFY	Wraps `HB_BUFFER_FLAG_VERIFY`.
Buffer.FLAG_PRODUCE_UNSAFE_TO_CONCAT	Wraps `HB_BUFFER_FLAG_PRODUCE_UNSAFE_TO_CONCAT`.
Buffer.FLAG_PRODUCE_SAFE_TO_INSERT_TATWEEL	Wraps `HB_BUFFER_FLAG_PRODUCE_SAFE_TO_INSERT_TATWEEL`.
Buffer.FLAG_DEFINED	Wraps `HB_BUFFER_FLAG_DEFINED`.

Glyph Flags

Buffer.GLYPH_FLAG_UNSAFE_TO_BREAK	Wraps `HB_GLYPH_FLAG_UNSAFE_TO_BREAK`.
Buffer.GLYPH_FLAG_UNSAFE_TO_CONCAT	Wraps `HB_GLYPH_FLAG_UNSAFE_TO_CONCAT`.
Buffer.GLYPH_FLAG_SAFE_TO_INSERT_TATWEEL	Wraps `HB_GLYPH_FLAG_SAFE_TO_INSERT_TATWEEL`.
Buffer.GLYPH_FLAG_DEFINED	Wraps `HB_GLYPH_FLAG_DEFINED`.

Class Feature

Feature.new (feature_string)	Wraps `hb_feature_from_string`
Feature:__tostring ()	Wraps `hb_feature_to_string`.

Class Variation

Variation.new (variation_string)	Wraps `hb_variation_from_string`.
Variation:__tostring ()	Wraps `hb_variation_to_string`.

Class Tag

Tag.new (string)	Wraps `hb_tag_from_string`.
Tag:__tostring ()	Wraps `hb_tag_to_string`.
Tag:__eq ()	Enables equality comparisions with == between two tags.

Class Script

Script.new (script)	Wraps `hb_script_from_string`.
Script.from_iso15924_tag (tag)	Wraps `hb_script_from_iso15924_tag`
Script:to_iso15924_tag ()	Wraps `hb_script_to_iso15924_tag`.
Script:__tostring ()	Enable nice output with `tostring(…)`
Script:__eq ()	Enables equality comparisions with == between two scripts.

Predefined Script Codes

Script.COMMON	Wraps `HB_SCRIPT_COMMON`.
Script.INHERITED	Wraps `HB_SCRIPT_INHERITED`.
Script.UNKNOWN	Wraps `HB_SCRIPT_UNKNOWN`.
Script.INVALID	Wraps `HB_SCRIPT_INVALID`.

Class Direction

Direction.new (dir)	Wraps `hb_direction_from_string`.
Direction:__tostring ()	Wraps `hb_direction_to_string`.
Direction:__eq ()	Enables equality comparisions with == between two directions.
Direction:is_valid ()	Wraps `HB_DIRECTION_IS_VALID`.
Direction:is_horizontal ()	Wraps `HB_DIRECTION_IS_HORIZONTAL`.
Direction:is_vertical ()	Wraps `HB_DIRECTION_IS_VERTICAL`.
Direction:is_forward ()	Wraps `HB_DIRECTION_IS_FORWARD`.
Direction:is_backward ()	Wraps `HB_DIRECTION_IS_BACKWARD`.

Predefined directions

Direction.LTR Wraps `HB_DIRECTION_LTR`.
Direction.RTL Wraps `HB_DIRECTION_RTL`.
Direction.TTB Wraps `HB_DIRECTION_TTB`.
Direction.BTT Wraps `HB_DIRECTION_LTR`.

Class Language

Language.new (lang)	Wraps `hb_language_from_string`.
Language:__tostring ()	Wraps `hb_language_to_string`.
Language:__eq ()	Enables equality comparisions with `==` between two languages.

Predefined languages

Language.INVALID Wraps `HB_LANGUAGE_INVALID`.

Class Set

Set:add (codepoint) Wraps `hb_set_add`.

Unicode functions

unicode.script (char) Wraps `hb_unicode_script`

Predefined Name IDs

ot.NAME_ID_COPYRIGHT	Wraps `HB_OT_NAME_ID_COPYRIGHT`
ot.NAME_ID_FONT_FAMILY	Wraps `HB_OT_NAME_ID_FONT_FAMILY`
ot.NAME_ID_FONT_SUBFAMILY	Wraps `HB_OT_NAME_ID_FONT_SUBFAMILY`
ot.NAME_ID_UNIQUE_ID	Wraps `HB_OT_NAME_ID_UNIQUE_ID`
ot.NAME_ID_FULL_NAME	Wraps `HB_OT_NAME_ID_FULL_NAME`
ot.NAME_ID_VERSION_STRING	Wraps `HB_OT_NAME_ID_VERSION_STRING`
ot.NAME_ID_POSTSCRIPT_NAME	Wraps `HB_OT_NAME_ID_POSTSCRIPT_NAME`
ot.NAME_ID_TRADEMARK	Wraps `HB_OT_NAME_ID_TRADEMARK`
ot.NAME_ID_MANUFACTURER	Wraps `HB_OT_NAME_ID_MANUFACTURER`
ot.NAME_ID_DESIGNER	Wraps `HB_OT_NAME_ID_DESIGNER`
ot.NAME_ID_DESCRIPTION	Wraps `HB_OT_NAME_ID_DESCRIPTION`
ot.NAME_ID_VENDOR_URL	Wraps `HB_OT_NAME_ID_VENDOR_URL`
ot.NAME_ID_DESIGNER_URL	Wraps `HB_OT_NAME_ID_DESIGNER_URL`
ot.NAME_ID_LICENSE	Wraps `HB_OT_NAME_ID_LICENSE`
ot.NAME_ID_LICENSE_URL	Wraps `HB_OT_NAME_ID_LICENSE_URL`
ot.NAME_ID_TYPOGRAPHIC_FAMILY	Wraps `HB_OT_NAME_ID_TYPOGRAPHIC_FAMILY`
ot.NAME_ID_TYPOGRAPHIC_SUBFAMILY	Wraps `HB_OT_NAME_ID_TYPOGRAPHIC_SUBFAMILY`
ot.NAME_ID_MAC_FULL_NAME	Wraps `HB_OT_NAME_ID_MAC_FULL_NAME`
ot.NAME_ID_SAMPLE_TEXT	Wraps `HB_OT_NAME_ID_SAMPLE_TEXT`
ot.NAME_ID_CID_FINDFONT_NAME	Wraps `HB_OT_NAME_ID_CID_FINDFONT_NAME`
ot.NAME_ID_WWS_FAMILY	Wraps `HB_OT_NAME_ID_WWS_FAMILY`
ot.NAME_ID_WWS_SUBFAMILY	Wraps `HB_OT_NAME_ID_WWS_SUBFAMILY`
ot.NAME_ID_LIGHT_BACKGROUND	Wraps `HB_OT_NAME_ID_LIGHT_BACKGROUND`
ot.NAME_ID_DARK_BACKGROUND	Wraps `HB_OT_NAME_ID_DARK_BACKGROUND`
ot.NAME_ID_VARIATIONS_PS_PREFIX	Wraps `HB_OT_NAME_ID_VARIATIONS_PS_PREFIX`
ot.NAME_ID_INVALID	Wraps `HB_OT_NAME_ID_INVALID`
ot.LAYOUT_NO_SCRIPT_INDEX	Wraps `HB_OT_LAYOUT_NO_SCRIPT_INDEX`
ot.LAYOUT_NO_FEATURE_INDEX	Wraps `HB_OT_LAYOUT_NO_FEATURE_INDEX`
ot.LAYOUT_DEFAULT_LANGUAGE_INDEX	Wraps `HB_OT_LAYOUT_DEFAULT_LANGUAGE_INDEX`
ot.LAYOUT_NO_VARIATIONS_INDEX	Wraps `HB_OT_LAYOUT_NO_VARIATIONS_INDEX`

Functions

version ()
Wraps `hb_version`.
shape (font, buffer[, options])

Wraps `hb_shape`.

Parameters:

- font `Font` to use for shaping
- buffer `Buffer` to shape
- options

  table containing one or more supported options:

  - `direction`: A `Direction` object representing the object.
  - `script`: A `Script` object representing the script.
  - `language`: A `Language` object representing the language.
  - `features`: features to enable, specified as either of the following.
    - comma-separated list of features. See feature string syntax reference
    - table of `Feature` objects

  (*optional*)

Class Blob

Lua wrapper for `hb_blob_t` type. Wraps binary data such as font file contents or raw font table data.

Blob.new (data)
Wraps `hb_blob_create`. Initializes a new `hb_blob_t`.

Parameters:

- data lua string containing binary or character data.

Returns:

a `Blob` object holding the binary data.

Blob.new_from_file (filename)
Wraps `hb_blob_create_from_file`. Initializes a new `hb_blob_t`.

Parameters:

- filename lua string.

Returns:

a `Blob` object holding the file contents.

Blob:get_length ()
Wraps `hb_blob_get_length`.

Returns:

length of the blob in bytes.

Blob:get_data ()
Wraps `hb_blob_get_data`.

Returns:

blob contents as a string.

Class Face

Lua wrapper for `hb_face_t` type

Face.new_from_blob (blob[, font_index=0])
Wraps `hb_face_create`. Initializes a new `hb_face_t` from a `Blob` object.

Parameters:

- blob `Blob` to read the font from.
- font_index index of font to read. (*default* 0)

Returns:

a `Face` object, or `nil` if the blob is invalid.

Face.new (file[, font_index=0])
Create a new `Face` from a file. Makes a call to `Face:new_from_blob` after creating a `Blob` from the file contents.

Parameters:

- file path to font file.
- font_index index of font to read. (*default* 0)

Returns:

a `Face` object, or `nil` if the file cannot be read.

Face:blob ()

Wraps `hb_face_reference_blob`.

Returns:

`Blob` containing the raw font data, or `nil`.

Face:get_name (name_id[, lang])

Wraps `hb_ot_name_get_utf8`.

Parameters:

- name_id an `ot.NAME_ID_*` constant.
- lang a `Language` object; uses default language if omitted. (*optional*)

Returns:

name string, or `nil` if not found.

Face:collect_unicodes ()

Wraps `hb_face_collect_unicodes`.

Returns:

table of codepoints supported by the face.

Face:get_glyph_count ()

Wraps `hb_face_get_glyph_count`.

Returns:

number of glyphs in the face.

Face:get_table (tag)

Wraps `hb_face_reference_table`.

Parameters:

- tag `Tag` object of the table.

Returns:

`Blob` object for the face table of `tag`.

Face:get_table_tags ()

Wraps `hb_face_get_table_tags`.

Returns:

table of `Tag`s representing face table tags.

Face:get_upem ()

Wraps `hb_face_get_upem`.

Returns:

units-per-em value of the face.

Face:ot_color_has_palettes ()

Wraps `hb_ot_color_has_palettes`.

Returns:

boolean indicating whether the face has color palettes.

Face:ot_color_palette_get_count ()

Wraps `hb_ot_color_palette_get_count`.

Returns:

number of color palettes in the face.

Face:ot_color_palette_get_colors ([palette_index=1])

Wraps `hb_ot_color_palette_get_colors`.

Parameters:

- palette_index 1-indexed palette index. (*default* 1)

Returns:

table of color tables each with `red`, `green`, `blue`, `alpha` fields, or `nil`.

Face:ot_color_has_layers ()
Wraps `hb_ot_color_has_layers`.

Returns:

boolean indicating whether the face has layered color glyphs.

Face:ot_color_glyph_get_layers (glyph)
Wraps `hb_ot_color_glyph_get_layers`.

Parameters:

- glyph glyph index.

Returns:

table of layer tables each with `glyph` and `color_index` fields, or `nil`.

Face:ot_color_has_png ()
Wraps `hb_ot_color_has_png`.

Returns:

boolean indicating whether the face has PNG color glyphs.

Face:ot_color_has_svg ()
Wraps `hb_ot_color_has_svg`.

Returns:

boolean indicating whether the face has SVG color glyphs.

Face:ot_color_glyph_get_svg (glyph)
Wraps `hb_ot_color_glyph_reference_svg`.

Parameters:

- glyph glyph index.

Returns:

`Blob` containing the SVG data, or `nil`.

Face:ot_var_has_data ()
Wraps `hb_ot_var_has_data`.

Returns:

boolean indicating whether the face has OpenType variation data.

Face:ot_var_find_axis_info (tag)
Wraps `hb_ot_var_find_axis_info`.

Parameters:

- tag `Tag` of the variation axis.

Returns:

table with fields `axis_index`, `tag`, `name_id`, `flags`, `min_value`, `default_value`, `max_value`, or `nil` if not found.

Face:ot_var_get_axis_infos ([start=1[, stop=-1]])
Wraps `hb_ot_var_get_axis_infos`.

Parameters:

- start 1-indexed start axis. (*default* 1)
- stop 1-indexed end axis (inclusive); `-1` means last. (*default* -1)

Returns:

table of axis info tables, each with fields `axis_index`, `tag`, `name_id`, `flags`, `min_value`, `default_value`, `max_value`.

Face:ot_var_named_instance_get_infos ([start=1[, stop=-1]])
Wraps `hb_ot_var_get_named_instance_count` and related functions.

Parameters:

- start 1-indexed start instance. (*default* 1)
- stop 1-indexed end instance (inclusive); `-1` means last. (*default* -1)

Returns:

table of instance info tables, each with fields `index`, `subfamily_name_id`, `postscript_name_id`.

Face:ot_var_named_instance_get_design_coords (index)

Wraps `hb_ot_var_named_instance_get_design_coords`.

Parameters:

- index 1-indexed named instance index.

Returns:

multiple return values, one number per axis.

Face:ot_var_normalize_variations (...)

Wraps `hb_ot_var_normalize_variations`.

Parameters:

- ... one or more `Variation` objects.

Returns:

multiple return values, one normalized integer coordinate per axis.

Face:ot_var_normalize_coords (...)

Wraps `hb_ot_var_normalize_coords`.

Parameters:

- ... one or more design-space coordinate numbers, one per axis.

Returns:

multiple return values, one normalized integer coordinate per axis.

Face:ot_layout_get_script_tags (table_tag)

Wraps `hb_ot_layout_table_get_script_tags`.

Parameters:

- table_tag `Tag` for the layout table (`GSUB` or `GPOS`).

Returns:

table of script `Tag`s, or `nil`.

Face:ot_layout_get_language_tags (table_tag, script_index)

Wraps `hb_ot_layout_script_get_language_tags`.

Parameters:

- table_tag `Tag` for the layout table (`GSUB` or `GPOS`).
- script_index index of the script.

Returns:

table of language `Tag`s, or `nil`.

Face:ot_layout_get_feature_tags (table_tag, script_index, language_index)

Wraps `hb_ot_layout_language_get_feature_tags`.

Parameters:

- table_tag `Tag` for the layout table (`GSUB` or `GPOS`).
- script_index index of the script.
- language_index index of the language.

Returns:

table of feature `Tag`s, or `nil`.

Face:ot_layout_find_script (table_tag, script_tag)

Wraps `hb_ot_layout_table_find_script`.

Parameters:

- table_tag `Tag` for the layout table (`GSUB` or `GPOS`).
- script_tag `Tag` of the script to find.

Returns:

boolean indicating whether the script was found, and its index.

Face:ot_layout_find_language (table_tag, script_index, language_tag)

Wraps `hb_ot_layout_script_find_language`.

Parameters:

- table_tag `Tag` for the layout table (`GSUB` or `GPOS`).
- script_index index of the script.
- language_tag `Tag` of the language to find.

Returns:

boolean indicating whether the language was found, and its index.

Face:ot_layout_find_feature (table_tag, script_index, language_index, feature_tag)

Wraps `hb_ot_layout_language_find_feature`.

Parameters:

- table_tag `Tag` for the layout table (`GSUB` or `GPOS`).
- script_index index of the script.
- language_index index of the language.
- feature_tag `Tag` of the feature to find.

Returns:

boolean indicating whether the feature was found, and its index.

Class Font

Lua wrapper for `hb_font_t` type

Font.new (face)

Wraps `hb_font_create`, and sets up some defaults for scale and shaping functions. Initializes a new `hb_font_t` from a `Face` object. Sets the default scale to the face's upem value, and sets the font shaping functions by calling `hb_ot_font_set_funcs` on it.

Parameters:

- face `Face` object.

Returns:

a `Font` object.

Font:get_scale ()

Wraps `hb_font_get_scale`.

Returns:

two values for the x-scale and y-scale of the font.

Font:set_scale (x_scale, y_scale)

Wraps `hb_font_set_scale`.

Parameters:

- x_scale desired x-scale of font.
- y_scale desired y-scale of font.

Font:get_h_extents ()

Wraps `hb_font_get_h_extents`.

Returns:

font extents table for horizontal direction, contains the following or `nil` if HarfBuzz fails to load font extents:

- `ascender`: typographic ascender.
- `descender`: typographic descender.
- `line_gap`: line spacing gap.

Font:get_v_extents ()

Wraps `hb_font_get_v_extents`.

Returns:

font extents table for vertical direction, similar to `Font:get_h_extents`, or `nil` if HarfBuzz fails to load font extents:

Font:get_glyph_extents (glyph)

Wraps `hb_font_get_glyph_extents`.

Parameters:

- glyph index inside the font.

Returns:

extents table contains the following or `nil` if HarfBuzz fails to load glyph extents:

- `x_bearing`: left side of glyph from origin.
- `y_bearing`: top side of glyph from origin.
- `width`: distance from left to right side.
- `height`: distance from top to bottom side.

Font:get_glyph_name (glyph)

Wraps `hb_font_get_glyph_name`.

Parameters:

- glyph index inside the font.

Returns:

name of the glyph or nil.

Font:get_glyph_from_name (name)

Wraps `hb_font_get_glyph_from_name`.

Parameters:

- name of the glyph.

Returns:

glyph index inside the font or nil.

Font:get_glyph_h_advance (glyph)

Wraps `hb_font_get_glyph_h_advance`.

Parameters:

- glyph index inside the font.

Returns:

advance glyph advance of the glyph in horizontal direction.

Font:get_glyph_v_advance (glyph)

Wraps `hb_font_get_glyph_v_advance`.

Parameters:

- glyph index inside the font.

Returns:

advance glyph advance of the glyph in vertical direction.

Font:get_nominal_glyph (codepoint.)

Wraps `hb_font_get_nominal_glyph`.

Parameters:

- codepoint.

Returns:

glyph index or `nil` if `codepoint` is not supported by the font.

Font:style_get_value (tag)

Wraps `hb_style_get_value`. Available when HarfBuzz >= 3.

Parameters:

- tag a `Tag` object for the style axis (e.g. `wght`, `ital`).

Returns:

style value as a number.

Font:ot_color_glyph_get_png (glyph)

Wraps `hb_ot_color_glyph_get_png`.

Parameters:

- glyph glyph index.

Returns:

`Blob` containing the PNG data, or `nil`.

Font:ot_metrics_get_position (tag)

Wraps `hb_ot_metrics_get_position`.

Parameters:

- tag a `Tag` object for the metric.

Returns:

integer position value, or `nil` if not available.

Font:ot_metrics_get_variation (tag)

Wraps `hb_ot_metrics_get_variation`.

Parameters:

- tag a `Tag` object for the metric.

Returns:

number variation value.

Font:ot_metrics_get_x_variation (tag)

Wraps `hb_ot_metrics_get_x_variation`.

Parameters:

- tag a `Tag` object for the metric.

Returns:

integer x variation value.

Font:ot_metrics_get_y_variation (tag)

Wraps `hb_ot_metrics_get_y_variation`.

Parameters:

- tag a `Tag` object for the metric.

Returns:

integer y variation value.

Font:set_variations (...)

Wraps `hb_font_set_variations`.

Parameters:

- ... one or more `Variation` objects.

Font:set_var_coords_design (...)

Wraps `hb_font_set_var_coords_design`.

Parameters:

- ... one or more design-space coordinate numbers, one per axis.

Font:set_var_coords_normalized (...)

Wraps `hb_font_set_var_coords_normalized`.

Parameters:

- ... one or more normalized coordinate integers, one per axis.

Font:set_var_named_instance (index)

Wraps `hb_font_set_var_named_instance`.

Parameters:

- index 1-indexed named instance index.

Font:get_var_coords_normalized ()
Wraps `hb_font_get_var_coords_normalized`.

Returns:

multiple return values, one normalized integer coordinate per axis.

Class Buffer

Lua wrapper for `hb_buffer_t` type.

Buffer.new ()
Wraps `hb_buffer_create`.

Returns:

a `Buffer` object.

Buffer:add (codepoint, cluster)
Wraps `hb_buffer_add`.

Parameters:

- codepoint Unicode codepoint to add.
- cluster cluster index of the codepoint.

Buffer:add_utf8 (text[, item_offset=0[, item_length=-1]])
Wraps `hb_buffer_add_utf8`.

Parameters:

- text UTF8 encoded string.
- item_offset 0-indexed offset in `text`, from where to start adding. (*default* 0)
- item_length length to add from `item_offset`. `-1` adds till end of `text`. (*default* -1)

Buffer:add_codepoints (text[, item_offset=0[, item_length=-1]])
Wraps `hb_buffer_add_codepoints`.

Parameters:

- text table with codepoints as lua numbers.
- item_offset 0-indexed offset in `text`, from where to start adding. (*default* 0)
- item_length length to add from `item_offset`. `-1` adds till end of `text`. (*default* -1)

Buffer:set_direction (dir)
Wraps `hb_buffer_set_direction`.

Parameters:

- dir A `Direction` object.

Buffer:get_direction ()
Wraps `hb_buffer_get_direction`.

Returns:

A `Direction` object.

Buffer:set_script (script)
Wraps `hb_buffer_set_script`.

Parameters:

- script A `Script` object.

Buffer:get_script ()
Wraps `hb_buffer_get_script`.

Returns:

A `Script` object.

Buffer:set_language (lang)
Wraps `hb_buffer_set_language`.

Parameters:

- lang A `Language` object

Buffer:get_language ()

Wraps `hb_buffer_get_language`.

Returns:

A `Language` object

Buffer:set_flags (flags)
Wraps `hb_buffer_set_flags`.

Parameters:

- flags see Buffer Flags

Buffer:get_flags ()
Wraps `hb_buffer_get_flags`.

Returns:

see Buffer Flags

Buffer:set_cluster_level (level)
Wraps `hb_buffer_set_cluster_level`.

Parameters:

- level see Cluster Levels

Buffer:get_cluster_level ()
Wraps `hb_buffer_get_cluster_level`.

Returns:

see Cluster Levels

Buffer:set_invisible_glyph (glyph)
Wraps `hb_buffer_set_invisible_glyph`.

Parameters:

- glyph glyph index to use for invisible characters.

Buffer:get_invisible_glyph ()
Wraps `hb_buffer_get_invisible_glyph`.

Returns:

glyph index used for invisible characters.

Buffer:set_replacement_codepoint (codepoint)
Wraps `hb_buffer_set_replacement_codepoint`.

Parameters:

- codepoint replacement codepoint for invalid input.

Buffer:get_replacement_codepoint ()
Wraps `hb_buffer_get_replacement_codepoint`.

Returns:

replacement codepoint for invalid input.

Buffer:get_length ()
Wraps `hb_buffer_get_length`.

Returns:

number of items in the buffer.

Buffer:guess_segment_properties ()
Wraps `hb_buffer_guess_segment_properties`.

Buffer:reverse ()
Wraps `hb_buffer_reverse`.

Buffer:clear_contents ()
Wraps `hb_buffer_clear_contents`.

Buffer:reset ()
Wraps `hb_buffer_reset`.

Buffer:pre_allocate (size)
Wraps `hb_buffer_pre_allocate`.

Parameters:

- size number of items to pre-allocate.

Returns:

boolean indicating success.

Buffer:get_glyphs ()

Helper method to get shaped glyph data. Calls `hb_buffer_get_glyph_infos`, `hb_buffer_get_glyph_positions` and `hb_glyph_info_get_glyph_flags`, and assembles the data into a Lua table.

Returns:

table containing data for each glyph, in a nested table. Each nested table contains the following:

- `x_advance`: horizontal advance.
- `y_advance`: vertical advance.
- `x_offset`: horizontal displacement.
- `y_offset`: vertical displacement.
- `cluster`: glyph cluster index within input.
- `codepoint`: glyph index inside the font *(this field name is a bit misleading, but that's what Harfbuzz uses)*.
- `flags`: glyph flags

Cluster Levels

See Harfbuzz docs for more details about what each of these levels mean.

Buffer.CLUSTER_LEVEL_MONOTONE_GRAPHEMES
Wraps `HB_BUFFER_CLUSTER_LEVEL_MONOTONE_GRAPHEMES`.

Buffer.CLUSTER_LEVEL_MONOTONE_CHARACTERS
Wraps `HB_BUFFER_CLUSTER_LEVEL_MONOTONE_CHARACTERS`.

Buffer.CLUSTER_LEVEL_CHARACTERS
Wraps `HB_BUFFER_CLUSTER_LEVEL_CHARACTERS`.

Buffer.CLUSTER_LEVEL_DEFAULT
Wraps `HB_BUFFER_CLUSTER_LEVEL_DEFAULT`.

Buffer Flags

See Harfbuzz docs for more details about what each of these flags mean.

Buffer.FLAG_DEFAULT
Wraps `HB_BUFFER_FLAG_DEFAULT`.

Buffer.FLAG_BOT
Wraps `HB_BUFFER_FLAG_BOT`.

Buffer.FLAG_EOT
Wraps `HB_BUFFER_FLAG_EOT`.

Buffer.FLAG_PRESERVE_DEFAULT_IGNORABLES
Wraps `HB_BUFFER_FLAG_PRESERVE_DEFAULT_IGNORABLES`.

Buffer.FLAG_REMOVE_DEFAULT_IGNORABLES
Wraps `HB_BUFFER_FLAG_REMOVE_DEFAULT_IGNORABLES`.

Buffer.FLAG_DO_NOT_INSERT_DOTTED_CIRCLE
Wraps `HB_BUFFER_FLAG_DO_NOT_INSERT_DOTTED_CIRCLE`.

Buffer.FLAG_VERIFY
Wraps `HB_BUFFER_FLAG_VERIFY`.

Buffer.FLAG_PRODUCE_UNSAFE_TO_CONCAT
Wraps `HB_BUFFER_FLAG_PRODUCE_UNSAFE_TO_CONCAT`.

Buffer.FLAG_PRODUCE_SAFE_TO_INSERT_TATWEEL
Wraps `HB_BUFFER_FLAG_PRODUCE_SAFE_TO_INSERT_TATWEEL`.

Buffer.FLAG_DEFINED
Wraps `HB_BUFFER_FLAG_DEFINED`.

Glyph Flags

See Harfbuzz docs for more details about what each of these flags mean.

Buffer.GLYPH_FLAG_UNSAFE_TO_BREAK
Wraps `HB_GLYPH_FLAG_UNSAFE_TO_BREAK`.

Buffer.GLYPH_FLAG_UNSAFE_TO_CONCAT
Wraps `HB_GLYPH_FLAG_UNSAFE_TO_CONCAT`.

Buffer.GLYPH_FLAG_SAFE_TO_INSERT_TATWEEL
Wraps `HB_GLYPH_FLAG_SAFE_TO_INSERT_TATWEEL`.

Buffer.GLYPH_FLAG_DEFINED
Wraps `HB_GLYPH_FLAG_DEFINED`.

Class Feature

Lua wrapper for `hb_feature_t` type

Feature.new (feature_string)
Wraps `hb_feature_from_string`

Parameters:

- feature_string See feature string syntax reference

Feature:__tostring ()
Wraps `hb_feature_to_string`. Enables nice output with `tostring(…)`.

Class Variation

Lua wrapper for `hb_variation_t` type.

Variation.new (variation_string)
Wraps `hb_variation_from_string`.

Parameters:

- variation_string variation string, e.g. `"wght=700"`.

Returns:

a `Variation` object, or `nil` if the string is invalid.

Variation:__tostring ()
Wraps `hb_variation_to_string`. Enables nice output with `tostring(…)`.

Class Tag

Lua wrapper for `hb_tag_t` type.

Tag.new (string)
Wraps `hb_tag_from_string`.

Parameters:

- string to be converted to a `Tag` object.

Returns:

a `Tag` object.

Tag:__tostring ()
Wraps `hb_tag_to_string`. Enable nice output with `tostring(…)`.

Returns:

Returns a string representation for the tag object.

Tag:__eq ()
Enables equality comparisions with `==` between two tags.

Returns:

`true` or `false` depending on whether the two tags are equal.

Class Script

Lua wrapper for `hb_script_t` type.

Script.new (script)
Wraps `hb_script_from_string`.

Parameters:

- script 4-letter script code according to the ISO 15924 standard.

Returns:

a `Script` object.

Script.from_iso15924_tag (tag)
Wraps `hb_script_from_iso15924_tag`

Parameters:

- tag a `Tag` object representing a ISO 15924 script.

Script:to_iso15924_tag ()
Wraps `hb_script_to_iso15924_tag`.

Returns:

a `Tag` object representing the script.

Script:__tostring ()
Enable nice output with `tostring(…)`

Returns:

Returns a 4-letter ISO 15924 script code for the script object.

Script:__eq ()
Enables equality comparisions with == between two scripts.

Returns:

`true` or `false` depending on whether the two scripts are equal.

Predefined Script Codes

Predefined directions that correspond to their original definitions in Harfbuzz.

Script.COMMON
Wraps `HB_SCRIPT_COMMON`.

Script.INHERITED
Wraps `HB_SCRIPT_INHERITED`.

Script.UNKNOWN
Wraps `HB_SCRIPT_UNKNOWN`.

Script.INVALID
Wraps `HB_SCRIPT_INVALID`.

Class Direction

Lua wrapper for `hb_direction_t` type.

Direction.new (dir)
Wraps `hb_direction_from_string`.

Parameters:

- dir can be one of `ltr`, `rtl`, `ttb`, `btt` or `invalid`.

Returns:

a `Direction` object.

Direction:__tostring ()
Wraps `hb_direction_to_string`. Enable nice output with `tostring(…)`.

Returns:

Returns a string representation for direction.

Direction:__eq ()
Enables equality comparisions with == between two directions.

Returns:

`true` or `false` depending on whether the two tags are equal.

Direction:is_valid ()
Wraps `HB_DIRECTION_IS_VALID`.

Returns:

a boolean value

Direction:is_horizontal ()
Wraps `HB_DIRECTION_IS_HORIZONTAL`.

Returns:

a boolean value

Direction:is_vertical ()
Wraps `HB_DIRECTION_IS_VERTICAL`.

Returns:

a boolean value

Direction:is_forward ()
Wraps `HB_DIRECTION_IS_FORWARD`.

Returns:

a boolean value

Direction:is_backward ()
Wraps `HB_DIRECTION_IS_BACKWARD`.

Returns:

a boolean value

Predefined directions

Predefined directions that correspond to their original definitions in Harfbuzz.

Direction.LTR
Wraps `HB_DIRECTION_LTR`.
Direction.RTL
Wraps `HB_DIRECTION_RTL`.
Direction.TTB
Wraps `HB_DIRECTION_TTB`.
Direction.BTT
Wraps `HB_DIRECTION_LTR`.

Class Language

Lua wrapper for `hb_language_t` type.

Language.new (lang)
Wraps `hb_language_from_string`.

Parameters:

- lang three-letter language tag to be converted to a `Language` object.

Returns:

a `Language` object.

Language:__tostring ()
Wraps `hb_language_to_string`. Enable nice output with `tostring(…)`.

Returns:

Returns a string representation for the language object.

Language:__eq ()
Enables equality comparisions with `==` between two languages.

Returns:

`true` or `false` depending on whether the two languages are equal.

Predefined languages

Predefined languages that correspond to their original definitions in Harfbuzz.

Language.INVALID
Wraps `HB_LANGUAGE_INVALID`.

Class Set

Lua wrapper for `hb_set_t` type.

Set objects represent a mathematical set of integer values.

Set:add (codepoint)
Wraps `hb_set_add`.

Adds a codepoint to the set.

Parameters:

- codepoint The element to add.

Unicode functions

unicode.script (char)
Wraps `hb_unicode_script`

Parameters:

- char Unicode codepoint

Returns:

a `Script` object.

Predefined Name IDs

Predefined OpenType 'name' table name identifier.

ot.NAME_ID_COPYRIGHT
Wraps `HB_OT_NAME_ID_COPYRIGHT`
ot.NAME_ID_FONT_FAMILY
Wraps `HB_OT_NAME_ID_FONT_FAMILY`
ot.NAME_ID_FONT_SUBFAMILY
Wraps `HB_OT_NAME_ID_FONT_SUBFAMILY`
ot.NAME_ID_UNIQUE_ID
Wraps `HB_OT_NAME_ID_UNIQUE_ID`
ot.NAME_ID_FULL_NAME
Wraps `HB_OT_NAME_ID_FULL_NAME`
ot.NAME_ID_VERSION_STRING
Wraps `HB_OT_NAME_ID_VERSION_STRING`
ot.NAME_ID_POSTSCRIPT_NAME
Wraps `HB_OT_NAME_ID_POSTSCRIPT_NAME`
ot.NAME_ID_TRADEMARK
Wraps `HB_OT_NAME_ID_TRADEMARK`
ot.NAME_ID_MANUFACTURER
Wraps `HB_OT_NAME_ID_MANUFACTURER`
ot.NAME_ID_DESIGNER
Wraps `HB_OT_NAME_ID_DESIGNER`
ot.NAME_ID_DESCRIPTION
Wraps `HB_OT_NAME_ID_DESCRIPTION`
ot.NAME_ID_VENDOR_URL
Wraps `HB_OT_NAME_ID_VENDOR_URL`
ot.NAME_ID_DESIGNER_URL
Wraps `HB_OT_NAME_ID_DESIGNER_URL`
ot.NAME_ID_LICENSE
Wraps `HB_OT_NAME_ID_LICENSE`
ot.NAME_ID_LICENSE_URL
Wraps `HB_OT_NAME_ID_LICENSE_URL`
ot.NAME_ID_TYPOGRAPHIC_FAMILY
Wraps `HB_OT_NAME_ID_TYPOGRAPHIC_FAMILY`
ot.NAME_ID_TYPOGRAPHIC_SUBFAMILY
Wraps `HB_OT_NAME_ID_TYPOGRAPHIC_SUBFAMILY`
ot.NAME_ID_MAC_FULL_NAME
Wraps `HB_OT_NAME_ID_MAC_FULL_NAME`
ot.NAME_ID_SAMPLE_TEXT
Wraps `HB_OT_NAME_ID_SAMPLE_TEXT`
ot.NAME_ID_CID_FINDFONT_NAME
Wraps `HB_OT_NAME_ID_CID_FINDFONT_NAME`
ot.NAME_ID_WWS_FAMILY
Wraps `HB_OT_NAME_ID_WWS_FAMILY`
ot.NAME_ID_WWS_SUBFAMILY
Wraps `HB_OT_NAME_ID_WWS_SUBFAMILY`
ot.NAME_ID_LIGHT_BACKGROUND
Wraps `HB_OT_NAME_ID_LIGHT_BACKGROUND`
ot.NAME_ID_DARK_BACKGROUND
Wraps `HB_OT_NAME_ID_DARK_BACKGROUND`
ot.NAME_ID_VARIATIONS_PS_PREFIX
Wraps `HB_OT_NAME_ID_VARIATIONS_PS_PREFIX`
ot.NAME_ID_INVALID
Wraps `HB_OT_NAME_ID_INVALID`
ot.LAYOUT_NO_SCRIPT_INDEX
Wraps `HB_OT_LAYOUT_NO_SCRIPT_INDEX`
ot.LAYOUT_NO_FEATURE_INDEX

Wraps `HB_OT_LAYOUT_NO_FEATURE_INDEX`

ot.LAYOUT_DEFAULT_LANGUAGE_INDEX

Wraps `HB_OT_LAYOUT_DEFAULT_LANGUAGE_INDEX`

ot.LAYOUT_NO_VARIATIONS_INDEX

Wraps `HB_OT_LAYOUT_NO_VARIATIONS_INDEX`